

# World-Model Based Reinforcement Learning for Hockey: DreamerV3 with Adaptive Opponent Curriculum

Carl Kueschall  
University of Tübingen

Winter Semester 2025/26

## 1 Introduction

*Note: Originally a two-person project; co-author exmatriculated. Work completed by remaining author.*

The hockey environment [2] presents a challenging continuous control problem in an adversarial setting. Two agents compete in an air-hockey game with 18-dimensional state observations encoding positions, velocities, and puck state, while each player controls 4 continuous action dimensions. Episodes last up to 250 timesteps with extremely sparse rewards: +10 for scoring, −10 for conceding, and 0 otherwise. This sparsity makes credit assignment difficult—agents must learn complex behaviors (positioning, puck control, shooting, defending) from reward signals that occur only at episode boundaries.

I implement DreamerV3 [1], a world-model based reinforcement learning algorithm that addresses sparse rewards by learning entirely in “imagination.” The agent first learns a world model from real experience, then trains its policy through simulated rollouts in learned latent space. My implementation builds upon the NaturalDreamer codebase<sup>1</sup>, with significant modifications for hockey: Two-Hot Symlog for sparse reward prediction, DreamSmooth [3] temporal reward smoothing, PFSP self-play, and an adaptive opponent curriculum. A key project realization was that late-stage performance depends less on fixed bots and more on recursive opponent-pool management and curated league diversity.

**Contributions:** (1) DreamerV3 hockey implementation with Two-Hot Symlog and DreamSmooth; (2) adaptive opponent curriculum with effective compute split of roughly 20% anchor-only, 20% bootstrap self-play, and 60% league-dominant self-play; (3) robust checkpoint-selection tooling using worst-case and quantile metrics, not only mean win rate; (4) strong late checkpoints (556k/612k/634k) that consistently outperform older baselines in summit cross-play.

## 2 Methods

### 2.1 DreamerV3: World-Model Based RL

#### 2.1.1 Background

DreamerV3 [1] learns a world model that predicts future states and rewards from actions, then trains actor-critic policies entirely through imagined rollouts in learned latent space. This is particularly suited for sparse-reward environments: the world model propagates reward information backward through imagined trajectories, enabling credit assignment even when real rewards are rare. Our implementation adapts this for the hockey domain, addressing (1) extremely sparse goal rewards, (2) continuous 4D action space, and (3) adversarial dynamics.

---

<sup>1</sup><https://github.com/InexperiencedMe/NaturalDreamer>

### 2.1.2 World Model Architecture

The world model uses a Recurrent State Space Model (RSSM) combining deterministic and stochastic dynamics. The state  $s_t = (h_t, z_t)$  consists of a deterministic GRU hidden state  $h_t \in \mathbb{R}^{256}$  and stochastic latent  $z_t$  (16 categorical variables, 16 classes each). Dynamics:  $h_t = \text{GRU}(h_{t-1}, z_{t-1}, a_{t-1})$ ; prior  $\hat{z}_t \sim p_\theta(z_t|h_t)$ ; posterior  $z_t \sim q_\phi(z_t|h_t, \text{Enc}(o_t))$ . KL divergence with free nats thresholding trains the model.

**Two-Hot Symlog.** MSE fails on hockey’s sparse reward distribution  $\{-10, 0, +10\}$ . We use Two-Hot Symlog encoding [1]:  $\text{symlog}(x) = \text{sign}(x) \cdot \ln(|x| + 1)$ , discretizing into 255 bins. The two-hot encoding places mass on bins surrounding the target, enabling gradient flow. Applied to both reward and value prediction.

### 2.1.3 Key Modifications for Hockey

Table 1 summarizes our domain-specific modifications. **DreamSmooth** [3] propagates goal rewards backward:  $\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$  with  $\alpha = 0.5$ , addressing temporal credit assignment for goals at episode boundaries.

| Modification     | Problem Addressed               | Implementation                                                            |
|------------------|---------------------------------|---------------------------------------------------------------------------|
| Two-Hot Symlog   | MSE fails on sparse rewards     | 255 bins in symlog space                                                  |
| DreamSmooth      | Temporal credit assignment      | $\tilde{r}_t = \alpha r_t + (1 - \alpha)\tilde{r}_{t+1}$ , $\alpha = 0.5$ |
| Anchor Opponents | Prevent catastrophic forgetting | 10% weak + 10% strong in late phase                                       |
| Self-Play + PFSP | Limited diversity               | 48-checkpoint pool, variance-weighted PFSP, periodic insertion            |

**Table 1:** Key modifications from baseline DreamerV3 for the hockey domain.

### 2.1.4 Recursive Opponent-Pool Management

The final training policy is controlled by opponent sampling rather than reward shaping. Let  $\rho$  be the anchor probability; then

$$\mathbb{P}(\text{anchor}) = \rho, \quad \mathbb{P}(\text{self-play}) = 1 - \rho.$$

In late runs we set  $\rho = 0.2$ , yielding an empirical distribution close to 80/10/10 (self-play/weak/strong). This keeps fixed-bot competence while maximizing exposure to diverse, hard opponents.

To avoid selecting brittle checkpoints, I rank candidates with robustness-aware metrics from cross-play:

$$\text{RobustScore} = 0.60 \cdot \text{CP}_{\min} + 0.25 \cdot \text{Fixed}_{\text{combined}} + 0.15 \cdot \text{CP}_{\text{mean}},$$

and in later tooling also use  $\text{CP}_{q20}$  (20th percentile) for stability under non-transitive matchups.

### 2.1.5 Behavior Learning

**Actor:** Tanh-squashed Gaussian with  $\sigma_{\min} = 0.1$ . **Critic:** Two-Hot Symlog with slow EMA target ( $\tau = 0.98$ ). **Lambda returns:**  $\text{TD}(\lambda)$ ,  $\lambda = 0.95$ ,  $\gamma = 0.997$ . **Value normalization:** Percentile-based scaling (5th/95th) for advantages. **Actor loss:**

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{s \sim \text{imagine}} \left[ \frac{V^\lambda(s) - V(s)}{S} + \eta \cdot H[\pi(\cdot|s)] \right] \quad (1)$$

with  $\eta = 3 \times 10^{-4}$  fixed.

### 3 Experimental Evaluation

#### 3.1 From 2-Phase Baseline to Adaptive 3-Stage Curriculum

The early project used a 2-phase recipe (mixed fixed bots, then self-play) and reached a strong fixed-bot baseline at 336k. The final push revealed a clearer compute-allocation structure (Table 2): fixed-bot training is necessary but quickly saturates, while most late gains come from league-dominant self-play with curated opponents.

| Stage                       | Compute Share | Main Opponents                   | Purpose                                   |
|-----------------------------|---------------|----------------------------------|-------------------------------------------|
| A: Anchor competence        | ~20%          | Weak + strong fixed bots         | Learn fundamentals, stabilize world model |
| B: Bootstrap self-play      | ~20%          | Internal self-play + anchors     | Introduce adversarial adaptation          |
| C: Curated league self-play | ~60%          | 80% league, 10% weak, 10% strong | Robustness against diverse opponents      |

**Table 2:** Practical training-time allocation learned during the final push.

In the representative late run (`_weak_seed42_20260224_040000`, 474k→639k in 10.7h), opponent sampling matched target behavior: self-play ratio  $\approx 0.80$ , anchor ratio  $\approx 0.20$ , and balanced weak/strong anchors. The self-play pool grew to 48 tracked opponents. Pool metrics improved substantially: overall pool win rate reached 0.804 and newest-third win rate rose to 0.725, indicating continued progress against recent opponents.

#### 3.2 Summit Cross-Play and Checkpoint Selection

I evaluate checkpoints with matrix cross-play against fixed bots plus a curated opponent bank. Results show clear improvement over older baselines, but also non-transitive behavior: ranking between 556k/612k/634k changes with seed and opponent set, while all three outperform 357k and 474k.

| Matrix Snapshot    | Top CP | Robust Score | CP Worst | Notes                                         |
|--------------------|--------|--------------|----------|-----------------------------------------------|
| 2026-02-24 (09:27) | 556k   | 0.6670       | 0.5330   | 3-candidate set                               |
| 2026-02-24 (12:59) | 612k   | 0.7196       | 0.6000   | extended opponent set                         |
| 2026-02-24 (14:23) | 634k   | 0.6664       | 0.5330   | 6-candidate set (556/612/622/634 + baselines) |

**Table 3:** Late-stage summit matrix results (30 episodes per fixed and checkpoint matchup).

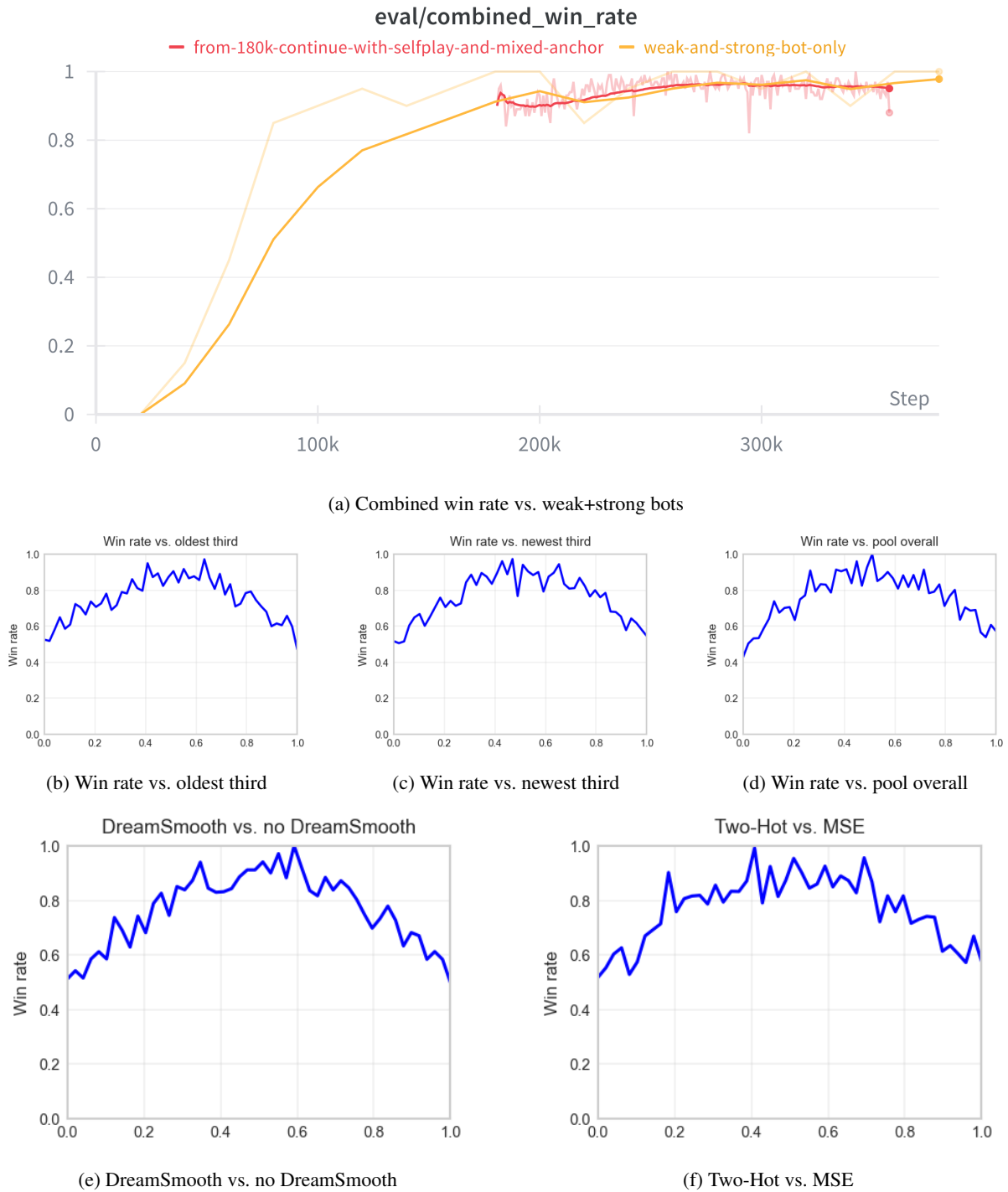
The main lesson is methodological: fixed-bot win rate saturates early and becomes weakly informative, while `CP_worst` and lower-quantile cross-play metrics identify robustness gaps and guide better pool curation. I therefore introduced automated shortlist selection (`eval_select_pool.py`) that ranks and greedily selects checkpoints by quality+diversity.

#### 3.3 Self-Play Metrics and Ablation Studies

Self-play progress remains visible in pool metrics (Figure 1, middle row), while ablations (bottom row) validate Two-Hot and DreamSmooth as enabling components for sparse reward learning.

#### 3.4 Fixed-Bot Benchmark Saturation

Table 4 shows that fixed-bot performance is high both for the earlier 336k benchmark and for late checkpoints, supporting the claim that fixed-bot metrics alone are no longer decisive in late training.



**Figure 1:** Top: Combined win rate vs. fixed bots across phases. Middle: Self-play metrics (win rate vs. pool thirds). Bottom: Ablation studies (DreamSmooth, Two-Hot). Placeholders replace with actual figures.

| Checkpoint      | Eval Setup       | Weak Win | Strong Win | Combined |
|-----------------|------------------|----------|------------|----------|
| 336k (baseline) | 100 eps/opponent | 90%      | 97%        | 93.5%    |
| 556k (late run) | 30 eps/opponent  | 96.7%    | 96.5%      | 96.5%    |
| 634k (late run) | 30 eps/opponent  | 96.7%    | 96.7%      | 96.7%    |

Late-run values from summit matrix snapshots; lower episode count than baseline benchmark.

**Table 4:** Fixed-bot performance saturates; cross-play robustness becomes the key discriminator.

## 4 Discussion

**Key Findings and Lessons Learned.** First, Two-Hot Symlog and DreamSmooth are necessary base components for sparse-reward hockey. Second, the dominant late-stage factor is opponent curriculum design: after fixed-bot competence is reached, most additional compute should be invested in self-play against a curated league. In my final runs, this effectively became a 20/20/60 allocation (anchor/bootstrap/curated-league) and produced stronger checkpoints than the earlier 2-phase baseline. Third, recursive opponent management (periodic checkpoint insertion, PFSP sampling, and matrix-based curation) is what unlocked peak performance, not further optimization on weak/strong bots.

**Limitations.** Matrix evaluations still use relatively small episode budgets in many snapshots (often 30), so ranking among close late checkpoints has variance. Non-transitivity remains visible, so a single “best” checkpoint may depend on opponent distribution. Some runs also revealed tooling constraints (e.g., incompatible legacy checkpoints) that required explicit filtering.

**Future Work.** Run higher-confidence bidirectional matrix evaluations (100+ episodes per matchup, multiple seeds), maintain a continuously refreshed league from multiple seeds, and integrate online tournament feedback into checkpoint promotion criteria.

## 5 AI Usage Declaration

**IDE Autocomplete:** Cursor AI used throughout. **Claude Code:** Learning (DreamerV3 internals, RL debugging) and routine tasks (plotting, W&B, evaluation scripts). Core algorithmic decisions from careful analysis.

## References

- [1] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2024.
- [2] G. Martius, P. Koley, M. Zhobro, and S. Ramen. Hockey environment. <https://github.com/martius-lab/hockey-env>, 2024.
- [3] V. Wu, J. Aslanides, J. Ba, and D. Hafner. Dreamsmooth: Improving model-based reinforcement learning via reward smoothing. *arXiv preprint arXiv:2311.01450*, 2023.